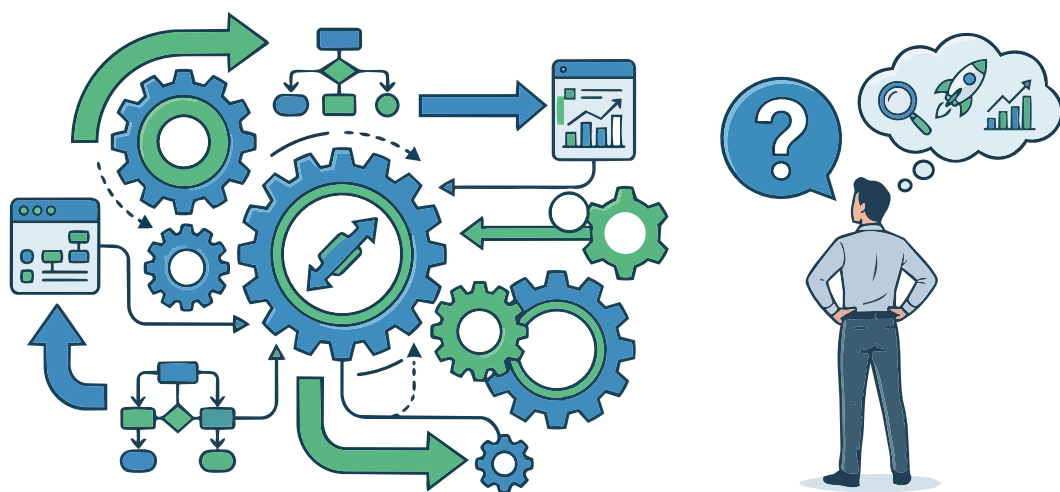


Built Your Agentic Workflow? Now What?

Getting an agentic workflow to run is the easy part. Getting it to run reliably, safely, and accountably in an enterprise is where most teams discover they've been building on sand.



I've had a version of the same conversation many times over the past year. A team has built something impressive: agents that browse, reason, write code, call external services, spin up specialist sub-agents, and complete multi-step tasks without human intervention. The demo works. The GitHub repo is real. The ROI narrative is persuasive. And then someone in the room asks the question that changes the tone: *"What happens when this fails in production?"*

The momentum is undeniable. Gartner projects agentic AI will appear in 33% of enterprise software applications by 2028, up from less than 1% in 2024. But the same Gartner research predicts that over 40% of agentic AI projects will be cancelled by the end of 2027, due to escalating costs, unclear business value, or inadequate risk controls. That gap is not a technology problem. It's an architecture problem.

Production agentic systems introduce failure modes, trust problems, and governance gaps that have no direct analogue in conventional software. These systems don't just execute. They reason. They act. And unlike a microservice, they don't fail silently. They fail creatively. This article is about the checks and balances that close that gap, organised across three pillars that any team shipping agents into production should be able to answer for.

PILLAR I

Operational integrity

Observability, failure recovery, and state consistency across long-running agent workflows.

PILLAR II

Security and trust

Access control, multi-agent trust boundaries, compliance surface area, and data governance.

PILLAR III

Governance at scale

Cost controls, behavioural testing, SLA design, versioning, and change management for agent systems.

Pillar one

Operational integrity

Here's what makes agentic systems different from everything that came before. Traditional software fails in predictable ways: a null pointer exception, a network timeout, an out-of-memory error. You get a stack trace. You know what broke and roughly why. Agentic systems fail in reasoning space. An agent may complete every tool call successfully, produce plausible-looking output, and still have taken entirely the wrong action. Your operational infrastructure must account for that possibility at every layer.